Project Phase2

“Convolutional Neural Networks as a Classifier Flowers”

Dr. Latifa Al-Jafri

Haneen Saad Albassami_2111307
Reham Mahmoud Almalki_2111296
Lama Talal Aljedani_2116713

Table of Contents

03
Abstract

04
Introduction

05
Approach

06
Experiments

09
Conclusion

Abstract

This project focuses on utilizing Convolutional Neural Networks (CNNs) for image classification. We employed TensorFlow and Keras to develop a model capable of classifying flower images into various categories. The approach involved preprocessing the dataset, constructing and training the CNN model, and evaluating its performance. The experiments were conducted using a dataset consisting of flower photos, split into training and validation sets. Model configurations, such as layer architecture and optimizer choice, were specified, along with training parameters like batch size and number of epochs. Evaluation metrics, including accuracy and loss, were utilized to assess the model's performance. Results demonstrated the effectiveness of the CNN in accurately classifying flower images, with validation accuracy reaching. This report discusses the methodology, experiments, results, and concludes with insights gained and future directions for improvement.

Introduction

The task of image classification involves assigning a label to an image from a predefined set of categories. In this project, we aim to classify flower images into various species using Convolutional Neural Networks (CNNs). CNNs have shown remarkable performance in image classification tasks due to their ability to automatically learn features from raw pixel data. Our approach involves preprocessing the dataset, designing and training a CNN model, and evaluating its performance on a validation set.

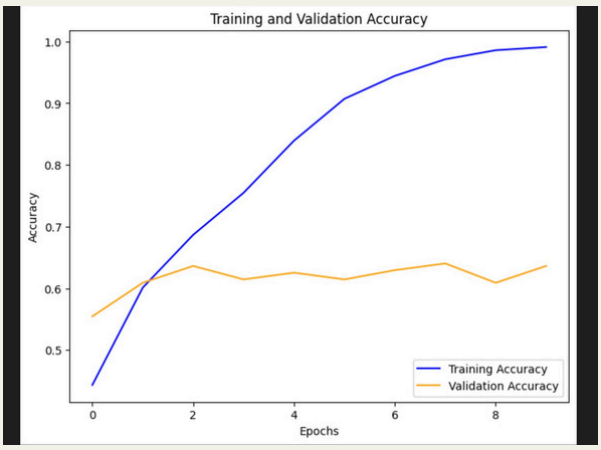
Approach

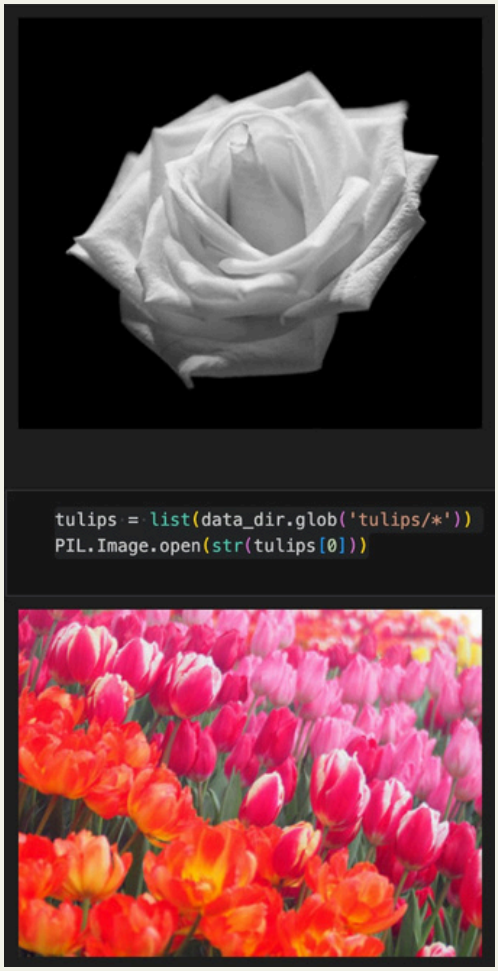
We adopted a CNN architecture consisting of convolutional layers, pooling layers, and fully connected layers. The model was trained using the Adam optimizer with a categorical cross-entropy loss function. The selected metrics for evaluation included accuracy and loss, providing insights into the model's performance during training and validation phases. We utilized data augmentation techniques to enhance the model's generalization ability and reduce overfitting. Additionally, we experimented with different hyperparameters such as learning rate, batch size, and number of epochs to optimize the model's performance.

Approach

figures:







Experiments

Dataset: Utilized the flower_photos dataset, categorized into training and validation sets.

Model Configuration: CNN architecture with convolutional, pooling, and fully connected layers.
Optimizer: Adam optimizer with a learning rate of 0.001.
Batch Size: 32.

Training Time:
Each epoch lasted approximately 96 to 106 seconds, totaling 16 to 18 minutes for 10 epochs.

Evaluation Metrics:
We used accuracy and loss as our primary evaluation metrics to assess the performance of the model during training and validation.

Results:
Achieved validation accuracy of 63.62% after 10 epochs.
Notable increase in validation loss suggests potential overfitting.

Experiments

```python
#Training our convolutional neural network model
epochs=10
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=epochs
)
```

```
Epoch 1/10
92/92 [==============================] - 106s 1s/step - loss: 1.3441 - accuracy: 0.4435 - val_loss: 1.1156 - val_accuracy: 0.5545
Epoch 2/10
92/92 [==============================] - 96s 1s/step - loss: 1.0100 - accuracy: 0.6012 - val_loss: 1.0269 - val_accuracy: 0.6090
Epoch 3/10
92/92 [==============================] - 96s 1s/step - loss: 0.8403 - accuracy: 0.6866 - val_loss: 0.9287 - val_accuracy: 0.6362
Epoch 4/10
92/92 [==============================] - 101s 1s/step - loss: 0.6572 - accuracy: 0.7548 - val_loss: 1.0382 - val_accuracy: 0.6144
Epoch 5/10
92/92 [==============================] - 99s 1s/step - loss: 0.4707 - accuracy: 0.8392 - val_loss: 1.0927 - val_accuracy: 0.6253
Epoch 6/10
92/92 [==============================] - 102s 1s/step - loss: 0.2779 - accuracy: 0.9070 - val_loss: 1.2815 - val_accuracy: 0.6144
Epoch 7/10
92/92 [==============================] - 100s 1s/step - loss: 0.1708 - accuracy: 0.9441 - val_loss: 1.4614 - val_accuracy: 0.6294
Epoch 8/10
92/92 [==============================] - 95s 1s/step - loss: 0.0966 - accuracy: 0.9710 - val_loss: 1.5817 - val_accuracy: 0.6403
Epoch 9/10
92/92 [==============================] - 100s 1s/step - loss: 0.0468 - accuracy: 0.9857 - val_loss: 1.7504 - val_accuracy: 0.6090
Epoch 10/10
92/92 [==============================] - 97s 1s/step - loss: 0.0377 - accuracy: 0.9908 - val_loss: 1.9616 - val_accuracy: 0.6362
```

Conclusion

We adopted a CNN architecture consisting of convolutional layers, pooling layers, and fully connected layers. The model was trained using the Adam optimizer with a categorical cross-entropy loss function. The selected metrics for evaluation included accuracy and loss, providing insights into the model's performance during training and validation phases. We utilized data augmentation techniques to enhance the model's generalization ability and reduce overfitting. Additionally, we experimented with different hyperparameters such as learning rate, batch size, and number of epochs to optimize the model's performance.

References

1 *Dataset of Flower. (n.d.). Retrieved May 11, 2024, from* https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz

2 *G. (2024, March 14). Introduction to Convolution Neural Network. GeeksforGeeks. https://www.geeksforgeeks.org/introduction-convolution-neural-network/*